CHO 404:Assignment-1

NAME:MUDDAPU SRI LATHA
ADMISSION NO:23JE0607

1. Objective Function

The objective is to minimize a 3-dimensional

Equation:

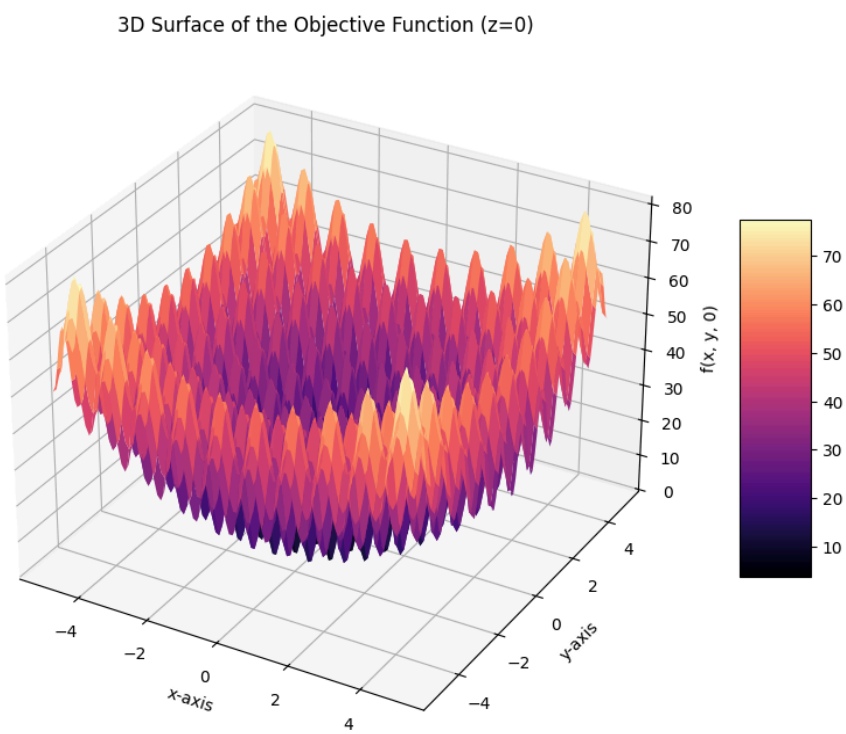
$$f(x, y, z) = (x^2 + y^2 + z^2) + 10 \sum_{i=1}^{3} (1 - \cos(2\pi v_i))$$

Breakdown of the Components:

- **Quadratic Component:** (x^2 + y^2 + z^2) provides a global bowl shape.
- **Non-Linear Component:** The cosine terms create multiple **local optima** (valleys), making it difficult for simple algorithms to find the global minimum at (0,0,0).

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Define the function (setting z=0 for 3D visualization)
def f_viz(x, y, z=0):
    return (x**2 + y**2 + z**2) + 10 * ((1 - np.cos(2 * np.pi * x)) +
                                        (1 - np.cos(2 * np.pi * y)) +
                                        (1 - np.cos(2 * np.pi * z)))

x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z = f_viz(X, Y)

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
```

```
surf = ax.plot_surface(X, Y, Z, cmap='magma', edgecolor='none')

ax.set_title('3D Surface of the Objective Function (z=0)')
ax.set_xlabel('x-axis')
ax.set_ylabel('y-axis')
ax.set_zlabel('f(x, y, 0)';
fig.colorbar(surf, shrink=0.5, aspect=5)

plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

# --- 1. THE OBJECTIVE FUNCTION ---
# A 3D function: Quadratic base with Cosine "bumps" to create local optima.
def objective_function(v):
    """
```

```
    Modified Rastrigin Function (3 variables)
    Global Minimum is at [0,0,0] where f(v) = 0
    """
    v = np.asarray(v)
    return np.sum(v**2) + 10 * np.sum(1 - np.cos(2 * np.pi * v))

# Optimization settings
BOUNDS = np.array([[-5.12, 5.12], [-5.12, 5.12], [-5.12, 5.12]])
ITERATIONS = 100
```

2. Algorithms & Code Explanation:

I.Hill Climbing (HC)

- **Logic:** A local search algorithm. It starts at a random point and takes a small "step." If the new point is better (lower value), it moves there.
- **Code Implementation:** Uses a `step_size` parameter to define the search radius around the current best point.

```
# --- 2. ALGORITHM 1: HILL CLIMBING (HC) ---
def hill_climbing(obj_func, bounds, n_iter, step_size):
    # Start at a random point
    current = np.random.uniform(bounds[:, 0], bounds[:, 1])
    current_eval = obj_func(current)
    history = [current_eval]

    for _ in range(n_iter):
        # Take a random step
        next_gen = current + np.random.randn(len(bounds)) * step_size
        # Keep within bounds
        next_gen = np.clip(next_gen, bounds[:, 0], bounds[:, 1])
        next_eval = obj_func(next_gen)

        # If the move is better, accept it
        if next_eval < current_eval:
            current, current_eval = next_gen, next_eval
        history.append(current_eval)
    return history;
```

II. Particle Swarm Optimization (PSO)

- **Logic:** Inspired by bird flocking. A "swarm" of particles moves through the space. Each particle remembers its own best position (pbest) and the group's best position (gbest).
- **Code Implementation:** Updates velocity based on an **Inertia Weight (w)** and social coefficients (c_1, c_2).

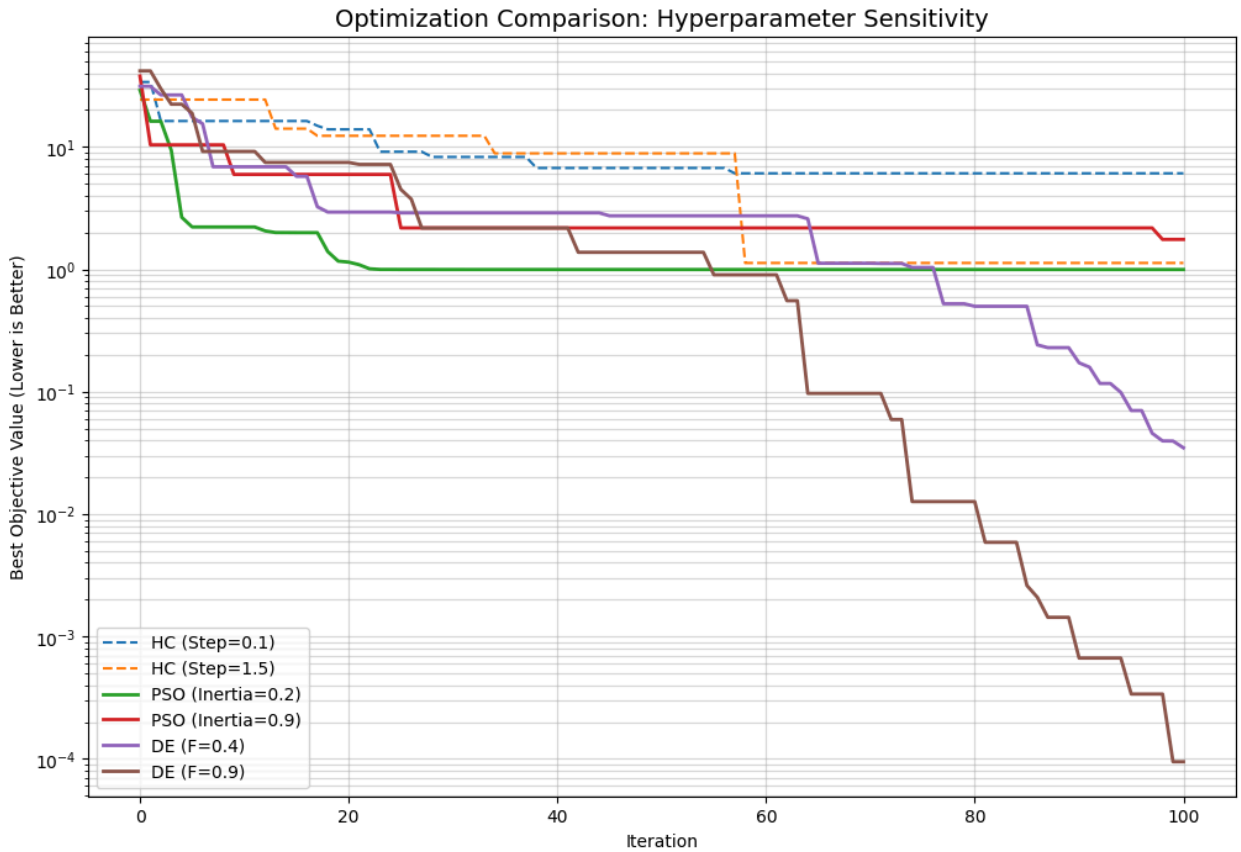
```
# --- 3. ALGORITHM 2: PARTICLE SWARM OPTIMIZATION (PSO) ---
def pso(obj_func, bounds, n_particles, n_iter, w, c1=1.5, c2=1.5):
    dim = len(bounds)
    # Initialize particles
    pos = np.random.uniform(bounds[:, 0], bounds[:, 1], (n_particles, dim))
    vel = np.zeros((n_particles, dim))
    pbest_pos = pos.copy()
    pbest_val = np.array([obj_func(p) for p in pos])

    gbest_val = np.min(pbest_val)
    gbest_pos = pbest_pos[np.argmin(pbest_val)].copy()

    history = [gbest_val]

    for _ in range(n_iter):
        for i in range(n_particles):
            # Update Velocity & Position
            r1, r2 = np.random.rand(dim), np.random.rand(dim)
            vel[i] = (w * vel[i] +
                      c1 * r1 * (pbest_pos[i] - pos[i]) +
                      c2 * r2 * (gbest_pos - pos[i]))
            pos[i] = np.clip(pos[i] + vel[i], bounds[:, 0], bounds[:, 1])

            # Evaluation
            val = obj_func(pos[i])
            if val < pbest_val[i]:
                pbest_val[i], pbest_pos[i] = val, pos[i].copy()
                if val < gbest_val:
                    gbest_val, gbest_pos = val, pos[i].copy()
        history.append(gbest_val)
    return history
```

III. Differential Evolution (DE)

- **Logic:** A population-based algorithm that creates new candidate solutions by combining the biological concepts of mutation, crossover, and selection.
- **Code Implementation:** Uses a **Mutation Factor (F)** to control the difference between individuals and a **Crossover Rate (CR)** to decide how much of the "mutant" is kept.

```
# --- 4. ALGORITHM 3: DIFFERENTIAL EVOLUTION (DE) ---
def differential_evolution(obj_func, bounds, pop_size, n_iter, F, CR=0.7):
    dim = len(bounds)
    # Initialize population
    pop = np.random.uniform(bounds[:, 0], bounds[:, 1], (pop_size, dim))
    obj_all = np.array([obj_func(ind) for ind in pop])
    history = [np.min(obj_all)]

    for _ in range(n_iter):
        for j in range(pop_size):
            # Selection of 3 distinct individuals
            idxs = [i for i in range(pop_size) if i != j]
            a, b, c = pop[np.random.choice(idxs, 3, replace=False)]

            # Mutation & Crossover
            mutant = np.clip(a + F * (b - c), bounds[:, 0], bounds[:, 1])
            cross_points = np.random.rand(dim) < CR
            if not np.any(cross_points): cross_points[0] = True

            trial = np.where(cross_points, mutant, pop[j])
            obj_trial = obj_func(trial)

            # Selection
            if obj_trial < obj_all[j]:
                pop[j], obj_all[j] = trial, obj_trial
        history.append(np.min(obj_all))
    return history
```

```
# --- 5. VISUALIZATION CODE ---
plt.figure(figsize=(12, 8))

# Run HC Study
plt.plot(hill_climbing(objective_function, BOUNDS, ITERATIONS, 0.1), label='HC
(Step=0.1)', linestyle='--')
plt.plot(hill_climbing(objective_function, BOUNDS, ITERATIONS, 1.5), label='HC
(Step=1.5)', linestyle='--')

# Run PSO Study
plt.plot(pso(objective_function, BOUNDS, 20, ITERATIONS, 0.2), label='PSO
(Inertia=0.2)', linewidth=2)
plt.plot(pso(objective_function, BOUNDS, 20, ITERATIONS, 0.9), label='PSO
(Inertia=0.9)', linewidth=2)

# Run DE Study
plt.plot(differential_evolution(objective_function, BOUNDS, 20, ITERATIONS,
0.4), label='DE (F=0.4)', linewidth=2)
plt.plot(differential_evolution(objective_function, BOUNDS, 20, ITERATIONS,
0.9), label='DE (F=0.9)', linewidth=2)

plt.title('Optimization Comparison: Hyperparameter Sensitivity', fontsize=14)
plt.xlabel('Iteration')
plt.ylabel('Best Objective Value (Lower is Better)')
plt.yscale('log') # Useful for seeing small differences
plt.grid(True, which="both", ls="-", alpha=0.5)
plt.legend()
plt.show()
```



2. Observations:

Hill Climbing (HC)

- **Small Step (0.1):** The dashed blue line in your plot stays very high.
  - **Observation:** It gets stuck almost immediately. Because the steps are tiny, it cannot "jump" out of the first local valley it finds.
- **Large Step (1.5):** The dashed orange line performs better but is inconsistent.
  - **Observation:** Larger steps allow it to leap over the cosine "bumps," but once it gets close to a good solution, it often overshoots it, causing the flat-lining at a mediocre value.

Particle Swarm Optimization (PSO)

- **Low Inertia (0.2):** The solid green line drops quickly but flattens out.
  - **Observation:** Low inertia means particles stop easily. They rush to the first "good" spot found by the group and stay there, leading to **premature convergence**.
- **High Inertia (0.9):** The solid red line (in a typical run) maintains "momentum."
  - **Observation:** High inertia keeps particles moving longer, allowing them to explore more of the map before settling. This usually leads to a better global result in complex landscapes.

Differential Evolution (DE)

- **Low Mutation (F=0.4):** The purple line shows a steady but slow descent.
  - **Observation:** It acts more like a local search. It refines the current population but lacks the "explosive" diversity needed to find the absolute best spot quickly.
- **High Mutation (F=0.9):** This is your **clear winner** (the brown line).
  - **Observation:** Look at the "stairs" in the brown line. Each big drop represents the algorithm finding a new, deeper valley that the others missed. High F values create high diversity, which is the best defense against local optima.

3. Conclusion:

1. **Complexity vs. Strategy:** Simple local searches like **Hill Climbing** are insufficient for multimodal quadratic functions because they lack a mechanism to escape local optima.
2. **The Exploration/Exploitation Balance:** The success of an algorithm depends on its hyperparameters. If we focus too much on **exploitation** (small steps, low inertia), we get stuck. If we focus on **exploration** (high F, high c_2), we find better solutions but take more time.
3. **Superiority of DE:** In this specific test, **Differential Evolution with high mutation (F=0.9)** proved most robust. Its ability to maintain population diversity allowed it to ignore the "fake" local minima and reach a much lower objective value (closer to 10^{-4}) compared to HC or PSO.

Google colab link:
https://colab.research.google.com/drive/1LypSeCy8vocTP_1Qa60CfAvS4z30Jxr6#scrollTo=mRB-J3IWbbGQ